

THREE COLORS OF MADNESS

Technical Design Document

Internal working document. Not for external distribution. Companion to the Design Bible.

Dejunai

Contents

Part One — What This Document Is

Part Two — Current Build

Part Three — Architecture As Built

Part Four — Architecture Decisions Needed

Part Five — Open Production Questions

Part Six — Next Steps

Part One — What This Document Is

This is the Technical Design Document (TDD) for Three Colors of Madness. It is the companion to the Design Bible, not a replacement for any part of it.

The division between the two documents is simple. The bible owns the what and the why — thesis, Design Laws, narrative, the meaning of every mechanic. This document owns the how — engine architecture, data schemas, production sequencing, and the specific open questions a playtest still needs to settle before a mechanism the bible describes gets its final shape. Where a passage in the bible states a mechanism rather than just a rule, that is deliberate: the mechanism itself is what's enforcing a Design Law, and this document does not relitigate it. Where the two documents ever appear to disagree, the bible is correct and this document is wrong until revised — never the reverse.

This document is expected to change often and change fast. Unlike the bible, it has no standing rule against being provisional — an architecture decision made here can be wrong, discovered to be wrong, and replaced without that being a failure of the document. Its job is to stay honest about the current state of the build, not to be permanently settled.

This revision grounds the document in a fourth pass over the same lineage: the same Godot branch, now carrying a further Codex pass on top of Claude Code's bug-fix content — three new content/correctness fixes, a genuine automated regression-test suite for two of them, and the branch's first playable Web export, which this revision verifies directly through a live browser session rather than by trusting the build's own self-report. Everything in Parts Two and Three below is a description of what the build actually does, verified by reading its source and commit history directly, or by playing the shipped Web build directly. Where something described in an earlier revision has since changed, been resolved, been superseded, or been corrected, that is called out explicitly rather than silently overwritten, so the document's own history of decisions and mistakes stays legible.

This revision also corrects a verification gap from v6, and independently checks a build-review document produced by a second coding agent (Antigravity) working from the same lineage while Codex was rate-limited. Both are called out in place below (Part Three and Part Four) rather than silently folded in, per this document's own practice of keeping its history of mistakes legible.

A second Antigravity delivery followed, implementing a substantial portion of that review's own Phase 1 and Phase 2 roadmap. This document verifies that delivery the same way it verifies everything else — direct source reading, not the delivery's own claims — and the results are folded into Parts Two through Four below.

A note on provenance, since two different coding agents now appear in this document's history: Codex is the project's core programmer of record, and this document's ordinary practice is to track Codex's own passes. Antigravity's work is secondary and experimental by default — it earns a place in this document only pass by pass, only after the kind of direct, independent source verification this document already applies to everything else, and only where that verification finds real, working, wired-in code rather than a claim. The Phase 1–2 delivery above cleared that bar: it is committed to the same git lineage Codex has been working from (confirmed by shared commit history, not a forked or divergent branch), it is reachable from the actual shipped scene rather than sitting in a dead prototype, and its test coverage asserts specific state this document read directly rather than took on faith. Being folded into this document is a statement that the work is real, not a statement that it is Codex's; where the two ever need distinguishing, Part Three names which agent produced which piece. When Codex resumes, its next pass should treat this delivery as its current baseline rather than reverting to an earlier checkpoint that doesn't know this work exists — the shared lineage makes that a continuation, not a merge.

Part Two — Current Build

Engine and Target

Godot 4.7 (the Web export identifies as 4.7.2.stable), using the GL Compatibility renderer (not Forward+), targeting broad hardware reach over cutting-edge rendering fidelity — confirmed directly in the Web build's own console output, which reports WebGL 2.0 via the Compatibility backend. Window target 1440×900, canvas-items stretch mode. No import dependencies beyond the engine itself. Unchanged since v1.

Scope of the Current Slice

The build still covers the opening movement of Chapter One ("No Exit Wound") — the Ophion estate grounds, Pickman Street, the precinct, the boardinghouse, Walter's room — and the service-passage encounter beneath the estate's kitchen wing described in this document's v2. This pass fixes three concrete correctness issues in content added by the prior pass, rather than adding new scenes.

- *Odell's answer can no longer be silently re-litigated.* `_odell_response()` now checks whether either of Odell's two recorded statements already exists before presenting the choice again; if one does, it shows an "Already answered" panel instead of the original two-button choice, so a player revisiting the captain after answering can't produce a second, contradictory statement. The prior build allowed the choice to be asked again indefinitely.
- *The woman outside Kessler's shop now actually leaves.* Her scene already stated, narratively, that she disappears after her warning; the code did not previously enforce that. A new `estate.gd/town.gd` method, `dismiss_old_woman()`, removes her interaction point and her figure once her scene is discovered, and is now called from every path that could otherwise re-show her — first discovery, save/load, and travel back into town. This is verified by new assertions in `tests/town_flow.gd` (see Verification, below) rather than left to observation.
- *The barman's "club talk" scene is retimed to match its own staging.* The scene was written as an evening bar conversation but is reached, in the current build, during a daytime portico encounter; the dialogue and its framing text ("BENEATH THE PORTICO" rather than "AT THE FAR END OF THE BAR") now describe the encounter that actually happens, and the question Walter asks shifted from present tense ("what the members talk about") to past tense ("what the members used to talk about") to match.

The build's own `docs/ARCHITECTURE.md` documents these three fixes plus two clarifications that were not code changes at all, just corrections to how the mechanism should be understood: the grounds crew (groundskeeper) observation added in the prior pass has no Perception gate and is available to any player regardless of accumulated evidence; and Perception, throughout the current build, gates *route information* (specifically, the wear-marks paragraph behind the tunnel's service screens) rather than

physical access — nothing in the current build blocks movement or a location outright behind a Perception check. Both are confirmed directly against source in this pass: `target("crew", ...)` in `estate.gd` is registered unconditionally alongside every other observation point, with no surrounding `if state.perception() >= N` guard, and the only `perception() >= 4` check in `scripts/chapters/chapter_one.gd` gates one paragraph of descriptive text, not a door, a route, or an interaction.

Not yet implemented, unchanged from v2: the glass-shattering ontological break, the fatal-comprehension ending, Chapters Two and Three, gamepad support, full key rebinding, a full encumbrance/leveling system, and Ekon's later consumption of Walter's record. The Part Seven difficulty screen and achievement registration are deferred. None of this pass's work touches the chapter's actual climax.

Two items leave that list this revision. The forced-spill flask mechanic and the tunnel combat/stagger system — both previously absent from the shipped game — are now implemented and wired into the live `chapter_one.gd/tunnel.gd` flow, verified directly against source rather than against either coding agent's own report. See the new Part Three subsection below for the detail.

(A note on the earlier "not yet implemented" entry for the forced-spill mechanic: v7 found a working prototype of it sitting unwired in `scripts/chapter_one_demo.gd`, an orphaned scene the shipped game never loads, and flagged it as possible prior art. The implementation that actually shipped this pass is a separate, fresh one built directly against `chapter_one.gd`; the orphaned demo prototype is untouched and still unwired. Worth a cleanup pass at some point, but it's dead code, not a duplicate system.)

Controls and Accessibility

Unchanged from v2.

Saves

Unchanged in mechanism from v2. The old woman's departure is itself now saved state — `dismiss_old_woman()` mutates the same `estate.points` dictionary and is re-applied on load rather than the removal being purely a runtime/session effect — so a player who saves after her scene and reloads does not see her reappear. No schema change to `case_state.gd` was needed.

Verification

This pass ships with genuine automated regression coverage for both of its behavioral fixes, not just its content. `tests/opening_flow.gd` now saves and reloads mid-test

specifically to assert Odell's locked answer survives a save/load round trip, and separately asserts that none of the dialogue buttons shown afterward begin with the original "Say nothing" option — the test fails if the lock can be bypassed by reopening the conversation. `tests/town_flow.gd` asserts the old woman's interaction point is gone immediately after her scene, gone again after an explicit save/load cycle, gone again after traveling away from and back into town, and that attempting to re-trigger her observation directly does not open a dialogue page. These are meaningfully specific assertions about state persistence, not smoke-test traversal. A `tests/capture_views.gd` addition also stages a dedicated camera position and lighting setup for a future "observer" QA screenshot — this pass adds the capture rig but does not include a rendered output image in this delivery; see Part Six.

Part Three — Architecture As Built

This section describes what the current codebase actually does, as read directly from the project and its commit history, plus what a live session against the shipped Web export actually showed. Where something differs from an earlier revision's Architecture As Built, that's noted explicitly; everything else in Part Three (autoloads, entry point/shared-module split, case-state shape, environment, the Observer color-preservation shader, dialogue architecture) is unchanged from v5 and not repeated here.

Web Export and Live Browser Verification — new

This is the branch's first commit to include an actual Web/WebAssembly export (`build/web/`), and this revision is the first to verify a build of this project by playing it directly, in a live browser session, rather than by reading source alone. The verification found:

- The title, accessibility-and-controls, chapter framing, and opening intertitle sequence all loaded and read exactly as the source predicts — including the "Eight people are dead at the Ophion estate. The town is prepared to account for six" framing and the prologue's "Those who came home disagreed about what they had seen" line, both consistent with Design Law 4.
- Movement, camera control, examine, and the case file all functioned correctly once reached in gameplay. The rose garden crime scene was reached and explored directly: the "Examine the six men" prompt, the named-victim breakdown (Judge Wexford, Dr. Fenn, Corliss the district attorney, Pruitt, Kessler, plus one unidentified sixth man), and Judge Wexford's specific examine text — "A wound above the bridge of the nose. No powder scorching. Walter turns the head. There is no exit wound." — all matched source exactly.

- Movement reliability was inconsistent earlier in this verification pass, in a way initially suspected to need a code-level fix (an input-latch on the movement controller, to guard against a held key being polled as released within a single physics frame). v6 searched this pass's commit history for that fix and found nothing, and concluded — wrongly, as corrected below — that no such change existed and that keeping the browser pane foregrounded was the real explanation.

This is a genuine milestone for verification going forward — future passes can be checked by playing the actual Web export rather than relying solely on source reading and the build's own self-report, the same standard already applied to git history in this document.

Correction to v6: the movement-latch fix does exist — new

v6 was wrong, and the error is instructive about the difference between a codebase's commit history and its actual delivered state. `main.gd` — the script the live `main.tscn` scene actually runs — carries exactly the fix v6 said didn't exist: a `MOVE_LATCH_MIN = 0.15` constant and a `move_latch_timer` dictionary, checked by a helper (`Input.is_action_pressed(action) or move_latch_timer[action] > 0.0`) and set on every `_input()` action-pressed event, with an explicit comment: "Guarantees a keydown/keyup pair resolves as movement even if it completes within one physics frame (synthetic/automated input)." That is precisely the fix this document's earlier session had hypothesized and then, on finding no supporting commit, explicitly declined to credit.

The reason `git log --all -p` turned up nothing is that the fix isn't committed. The delivered zip's working tree carries a substantial set of uncommitted changes against its own `HEAD` — `main.gd`, `case_state.gd`, `estate.gd`, `chapter_one.gd`, `tunnel.gd`, `town.gd`, `story.gd`, `film.gdshader`, and several test files all differ from the last commit — consistent with the zip having been made directly from a live local working directory rather than a clean checkout. v6's verification method (read the commit history) was sound for a normal git branch but blind to this specific delivery, which put its most relevant recent work in the working tree instead of a commit. Future verification passes against a zipped delivery should diff the working tree against `HEAD` (`git status`, `git diff HEAD`) in addition to reading commit history, specifically because a delivery is not guaranteed to be a clean checkout.

This does not change the underlying finding that browser-pane focus also mattered during testing — both things can be true, and likely are: a real input-latch fix landed in the working tree, and foregrounding the pane was still probably necessary for reasons outside the game's own code (the embedded browser's pointer/keyboard event delivery).

What changes is that this document was wrong to say no code-level fix exists. It does, and it is functioning as intended.

Forced-Spill, Tunnel Combat, and Generalized Systems (Antigravity Phase 1–2) — new

A second delivery from Antigravity implemented most of its own review's Phase 1 and Phase 2 roadmap. Checked directly against source rather than against the delivery's own report:

- `_town_observation()` is now genuinely generalized. Its second parameter is `return_target: Variant`, accepting a `Callable` (called directly on completion), a `bool` (backward-compatible shorthand for reopening `_witness_menu()`), or nothing (falls through to the ordinary close-and-toast). Father Behan's menu now passes `_behan_menu` itself as the return target instead of the inlined card/discover/reopen sequence v6 said a third menu would need. This resolves the item this document has carried since v5.
- A real, declarative NPC lifecycle system now exists in the base `estate.gd` class: `register_actor(id, node, target_id, condition)` stores a per-actor `Callable`; `sync_actors(state)` evaluates every registered actor's condition against current state and calls `dismiss_actor()` for any that resolve false, which erases the actor's interaction point and hides/frees its figure. `town.gd`'s old woman is migrated onto it — `register_actor("old_woman", old_woman, "old_woman", func(st): return not st.evidence.has("old_woman"))` — giving this document's long-standing "no general scheduling system" item its actual resolution rather than the single-NPC precedent v6 described. One leftover worth a cleanup pass: `town.gd`'s own `dismiss_old_woman()` still calls `dismiss_actor("old_woman")` and then redundantly hides/frees `departing_woman` itself — harmless (the node is already gone by then), but vestigial now that the generic system does the same work.
- The forced-spill flask mechanic is implemented and wired into the real flow. `_tunnel_descent()` in `chapter_one.gd` checks `state.flask_spilled`; on first entry it sets `flask_spill_amount = state.flask`, zeroes the flask, and plays a diegetic card ("TORN FROM THE STRAP" / "THE ROCK SPUR") with matching narration in `tunnel_story.gd`. The inspectable case-file flask entry states the exact amount lost, satisfying Design Law 4's redundant-tell requirement — confirmed by reading the actual label text, not by trusting a test's own assertion of it.
- Tunnel combat exists and matches the bible's asymmetry rule closely. Two encounters, `_cultist_encounter()` and `_drowned_encounter()` in

`chapter_one.gd`, backed by `cultist_stagger/drowned_stagger` timers in `tunnel.gd`. The cultist: a six-round revolver and Kessler's boning knife (stagger duration $1.5 + \text{strength} * 1.0$ for the knife), staggered only, with the line "No force can destroy it permanently" stated outright after every hit — and the cultist's "thin, wet cough" cue from the bible's Drowned section is the actual line played on encounter. The drowned sailor: the same two tools, a longer knife-stagger formula ($2.0 + \text{strength} * 1.5$, matching the bible's claim that Strength matters more here), and — once staggered — a "Deliver desperate finishing blow" option that sets `state.drowned_dead = true` permanently, with the text explicitly contrasting the two populations ("Different rules from the other thing... this body carries no curse; brutal force ends it permanently"), and the "low, impersonal groan" cue the bible specifies for this population, distinct from the cultist's cough.

- *A genuine death-and-reload fail state exists*, tied to the pre-existing stealth/exposure cycle rather than to ammo exhaustion specifically: `tick_world()` calls `estate.advance(delta, player.position)`, which returns true once exposure while spotted crosses the notice threshold, and that triggers `_show_tunnel_death()` — a real "Walter died in the passage" panel with a checkpoint-restore (`_retry_tunnel()`, backed by a saved `tunnel_checkpoint` snapshot), consistent with Design Law 11's death-and-reload requirement. This document did not find a separate fail state specifically for "out of ammo, no stagger banked, no way out," which the bible also describes; worth checking whether that's a real gap or simply subsumed by the exposure-death mechanic before treating it as open.
- *A new dedicated test file*, `tests/phase_two_mechanics.gd`, asserts real, specific state rather than traversal: the exact spilled amount, the inspectable flask label text containing it, ammo decrementing by exactly one round per shot, and the cultist's stagger timer going positive after a hit. The delivery's own `docs/qa/integration-test.log` reports both existing QA suites passing; this document did not independently execute the test suite (no Godot binary is available in this session's environment) and reports the log's content as self-reported, not verified firsthand — a distinction from the source-level checks above, which were read directly.
- *The observer QA screenshot still does not exist*. `tests/capture_views.gd` still stages a `capture_mode == "observer"` rig writing to `qa_observer.png`, and `docs/qa/` now holds several new renders (`qa_dialogue`, `qa_effects`, `qa_gate`, `qa_large_text`, `qa_world`, and four town captures) — but none of them is the observer/groundskeeper-accent capture this document has been asking for since v5. Still open; see Part Six.

Part Four — Architecture Decisions Needed

Items resolved since v5 are marked as such rather than deleted, so the decision history stays visible. Items unchanged from v5 are restated briefly for continuity; see v5's fuller reasoning if needed.

Odell's answer could be silently re-litigated — resolved. See Part Two.

`_odell_response()` now locks against either recorded statement.

The old woman outside Kessler's shop did not actually leave — resolved. See Part Two.

`dismiss_old_woman()` now enforces the departure the scene already narrated, across discovery, save/load, and travel.

The barman's club-talk scene was staged for the wrong time of day — resolved. See Part Two.

Shader color-preservation scope — unchanged from v5; still open. The active shader preserves color through a screen-wide red-dominance test, supported by a controlled art palette, with no object-specific mask. A capture rig for a dedicated "observer" QA screenshot was added this pass (`tests/capture_views.gd`), but no rendered output from it shipped in this delivery — see Part Six. True object-specific isolation remains a separate rendering decision if later assets or lighting make palette-based selection unreliable.

`_town_observation()`'s hardcoded return-to-witness argument — resolved. See Part Three. `return_target` now accepts a `Callable`; Behan's menu passes itself directly rather than inlining its own sequence.

NPCs have no general scheduling or staging trigger system — resolved. See Part Three.

`register_actor()/sync_actors()/dismiss_actor()` in `estate.gd` give any actor a declarative presence condition; the old woman is migrated onto it. The gardener and other always-present figures in `estate.gd`'s `_ready()` remain unconditional, which is correct — they have no presence rule to encode, not a gap. `town.gd`'s `dismiss_old_woman()` retains a redundant explicit hide/free call on top of the generic system's own; worth trimming, not worth blocking on.

Perception gates route information, not physical access — clarified, not new. The build's own `docs/ARCHITECTURE.md` states this explicitly as a correction to how the mechanism should be understood, and this document confirms it directly against source: the only `perception() >= 4` check in `chapter_one.gd` gates a single descriptive paragraph,

not a door, a route, or an interaction point. Nothing about how the current build actually behaves changed; only the risk of describing it as an access gate is now closed off.

Two coexisting case-tracking systems — still open, unchanged from v2. No change this pass.

Procedural geometry: blockout strategy or production pipeline? — still open, unchanged from v2. No change this pass.

Combat and the forced-spill mechanic — resolved. See Part Three. Both are implemented and wired into the shipped `chapter_one.gd/tunnel.gd` flow, not merely prototyped.

No dedicated fail state for "exhausted resources, no way out" in tunnel combat — new, still open. See Part Three. The only tunnel death path found is exposure during the stealth cycle. Whether the bible's exhausted-resources death (empty revolver, no stagger banked, cornered) is a real gap or intentionally subsumed by the exposure mechanic hasn't been settled.

Verifying a Second Agent's Build Review (Antigravity)

While Codex was rate-limited, a build-review and roadmap document was produced by a second coding agent (Antigravity), working from what it described as the current Codex codebase. Per this document's standing practice, its specific technical claims were checked directly against source rather than accepted on the review's own authority.

Confirmed accurate at the time: the corkboard's formula in `case_state.gd` is exactly `2 + mini(4, evidence.size() / 2)`, a flat count-based formula rather than the diegetic causal-spine-completion model Part Three of the bible describes — still true in this pass; see below. `_town_observation()`'s hardcoded return and the absence of a general NPC lifecycle system were also accurate at the time and are now both resolved, per Part Three above. No combat, stagger, revolver, melee, or population-specific rise cues existed at the time; the review's claim that Chapter One's combat and finishing-blow systems were entirely unbuilt held up then — a second delivery has since built them (Part Three). No glass-shatter/ontological-break system, difficulty screen, or achievement registration existed then and still don't now.

One claim the review overstated at the time: it listed the forced-spill flask mechanic as needing new implementation, when a working (if unwired) prototype already existed in `chapter_one_demo.gd`. The implementation that actually shipped is a fresh one against `chapter_one.gd`, not a port of that prototype — see Part Two.

The review's four-phase roadmap (architectural debt, then flask/corkboard/combat, then the climax and ending, then the Part Seven meta layer) has now had its first two phases

substantially delivered and verified; Phase 3 (the ontological break and ending) and Phase 4 (the meta layer) remain untouched.

The corkboard/causal-spine model — still open, unchanged. Untouched by either Antigravity delivery. Still the single most consequential open item carried over from v5.

Part Five — Open Production Questions

Unchanged from v5 in substance; nothing in this pass bears directly on Chapter Two's, Chapter Three's, or the meta layer's open questions.

Part Six — Next Steps

In rough priority order: resolve the corkboard/link-mechanic decision, still the single most consequential open item and untouched by either Antigravity pass. Build the ontological break — glass-shatter cue, aspect-ratio widening, color bleed, and the entity's Constance-shaped manifestation — and the fatal-comprehension ending; this is now the largest remaining piece of Chapter One's actual climax, with the town/estate content, the forced-spill mechanic, and tunnel combat all now in place around it. Capture and review the actual "observer" QA screenshot the capture rig in

`tests/capture_views.gd` was built to produce; it still does not exist despite several other QA renders shipping this pass, so the shader's real-world legibility against the current palette remains unconfirmed by any image. Decide whether the palette-discipline convention for color isolation needs to become an actual object/material mask before more Observer content is added. Decide the procedural-art-pipeline question before committing more chapters to code-generated geometry. Settle whether tunnel combat needs its own exhausted-resources death state or whether the exposure-death mechanic already covers it (see Part Four). Trim `town.gd`'s now-redundant explicit hide/free call in `dismiss_old_woman()` now that the generic lifecycle system handles it. Build the Part Seven difficulty screen and achievement registration once the climax exists to award them against. Extend the live-Web-export verification practice going forward, and continue diffing a zipped delivery's working tree against `HEAD` in addition to reading commit history — both practices caught real things in this document's own history that source reading alone, done the old way, had missed. Once a Chapter Two slice exists, this document gets a matching Architecture As Built entry for it, held to the shared/chapter-specific template v2 established.